김현수 Kim Hyunsoo

> 불편함을 이해하고, 더 쉬운 경험을 만드는 프론트엔드 개발자

Information

Email | kjy5662860@naver.com

Github | https://github.com/kjy5286

Education

중앙대학교 소프트웨어학부

2021.03 ~ 2027.02 졸업예정

평균평점 4.10 / 4.50

Certificates

정보처리기사

SQLD

Skills

Frontend | React, TypeScript, JavaScript, Vite
API/Auth | Axios, REST API, JWT, Secure Store
Mobile | React Native, Expo, FCM, EAS Build
UI/UX | Figma, User Flow, Responsive UI
Deploy | Vercel, Git, Android Test Build

Introduce

안녕하세요. 프론트엔드 개발자 김현수입니다.

사용자 흐름을 먼저 생각합니다.

서비스의 가치는 기능의 복잡함보다 사용자가 자연스럽게 목적을 달성할 수 있는 흐름에서 나온다고 생각합니다. 화면을 구현할 때도 단순히 보이는 요소를 만드는 데 그치지 않고, 사용자의 상황과 행동 순서, 필요한 피드백을 함께 고려합니다.

설계와 구현을 함께 고민합니다.

Figma로 화면 구조와 사용자 흐름을 먼저 정리하고, 이를 React 기반 웹과 모바일 화면으로 구현해왔습니다. 디자인 의도가 실제 기능으로 이어질 수 있도록 서버 데이터, API 응답, 예외 상황을 함께 확인하며 개발했습니다.

기술을 사용자에게 가까운 형태로 전달하고 싶습니다.

복잡한 기술도 사용자가 어렵게 느낀다면 좋은 서비스가 되기 어렵다고 생각합니다. 앞으로도 사용자의 문제를 이해하고, 이를 직관적인 화면과 안정적인 기능으로 연결하는 개발자가 되고 싶습니다.

Project

그리고너(DRAW&YOU) (2025.07 ~ 2025.12)

그림/일기를 기반으로 사용자의 감정 상태를 분석하고, 맞춤형 콘텐츠를 추천하는 **AI 심리 케어 웹 서비스**

사용자는 웹 캔버스에서 그림을 직접 그리거나 이미지를 업로드하고, 일기를 작성한 뒤 AI 분석 결과를 통해 감정 건강 점수, 스트레스 수준, 긍정성 지수, 총평을 확인할 수 있습니다.

GitHub | https://github.com/DRAW-AND-YOU/WEB-Frontend

Service | https://drawandyou.com

Figma | DRAW&YOU Wireframe

Manual | DRAW&YOU 사용자 메뉴얼

Team | Plan(1) / Frontend Developer(1) / Backend Developer(1) / AI Engineer(1)

사용기술

- React, Vite, JavaScript
- React Router, Axios
- react-konva, Konva
- Framer Motion
- Vercel, AWS S3 Image Upload Flow

핵심 구현 내용

- 홈·드로잉·다이어리·갤러리·분석 결과·마이페이지 구현
- 그림 및 일기 입력 → 분석 요청 → 결과 확인 → 추천 콘텐츠로 이어지는 흐름 구성
- AWS S3 이미지 업로드 흐름 연동, Vercel 배포, 사용자 매뉴얼 작성

1. 편집 상태를 관리하는 웹 드로잉 캔버스 구현

[문제 상황]

그림은 AI 분석의 입력 데이터이므로, 사용자가 웹에서 작성하고 수정할 수 있는 환경이 필요했습니다. 단순히 선을 그리는 기능에 더해 브러시·색상·지우개·되돌리기처럼 여러 편집 동작이 화면에 일관되게 반영되어야 했습니다. 직접 그리기와 기존 이미지 업로드도 각각의 입력 방식으로 지원해야 했습니다.

[해결 과정]

① 입력 방식을 분리하고 각각의 진입 흐름 구성

사용자가 웹에서 바로 그림을 작성하는 경우와 기존 이미지 파일을 사용하는 경우를 구분했습니다.

직접 그리기에는 편집 도구를 제공하고, 이미지 업로드에는 파일을 선택하는 흐름을 구성했습니다. 입력 방식은 달라도 그림을 준비한 뒤 분석을 요청한다는 동일한 서비스 흐름으로 연결했습니다.

② 선 입력을 React 상태로 관리하고 캔버스에 반영

사용자의 선 입력을 상태로 관리하고, react-konva와 Konva를 활용해 해당 데이터를 캔버스에 표시했습니다.

그림을 수정하는 기능이 필요했으므로, 그려진 화면뿐 아니라 수정 대상인 선 데이터도 관리했습니다. 편집 동작을 상태 변경과 화면 갱신으로 연결하는 구조를 구성했습니다.

③ 도구별 편집 동작 구현

브러시 굵기와 색상을 변경할 수 있도록 도구 옵션을 구성하고, 지우개로 그림을 수정하는 기능을 구현했습니다.

되돌리기·다시 실행과 캔버스 초기화를 제공해, 사용자가 작업 중 실수를 수정하거나 입력을 다시 시작할 수 있도록 했습니다.

④ 편집 화면을 분석 요청 흐름과 연결

그림을 작성하는 화면과 분석 결과를 확인하는 화면이 분리된 기능으로 끝나지 않도록, 입력 이후 분석 요청과 결과 확인으로 이어지는 사용 흐름을 구성했습니다.

[결과]

별도 그림 프로그램 없이 브라우저에서 그림을 작성·편집하고 분석을 요청할 수 있는 입력 환경을 구현했습니다. 캔버스 편집 기능과 이미지 업로드를 서비스의 분석 흐름에 연결했습니다.\

[배운점]

사용자가 직접 조작하는 도구형 UI에서는 기능의 개수만큼이나 입력 데이터와 화면 상태의 관계가 중요하다는 점을 배웠습니다. 편집 동작이 어떤 상태를 바꾸고 그 변화가 화면에 어떻게 반영되는지까지 고려해야 합니다.

2. AI 분석 결과의 정보 구조와 조회 흐름 설계

[문제 상황]

AI 분석 결과에는 감정 건강 점수, 스트레스 수준, 긍정성 지수, 총평과 추천 콘텐츠가 함께 포함됐습니다. 서로 성격이 다른 정보를 같은 수준으로 나열하면, 사용자가 자신의 상태를 먼저 파악하고 이후 추천 콘텐츠를 선택하는 흐름을 안내하기 어려웠습니다. 분석을 기다리는 시점과 결과를 다시 조회하는 시점도 고려해야 했습니다.

[해결 과정]

① 응답 정보를 사용자에게 필요한 순서로 구분

분석 응답에 포함된 감정 건강 점수·스트레스 수준·긍정성 지수, 총평, 장소·음악·영상 추천을 서로 다른 정보 묶음으로 정리했습니다.

자신의 상태를 파악한 뒤 설명을 읽고 추천 콘텐츠를 탐색할 수 있도록, 점수와 감정 지표를 먼저 확인하는 화면 위계를 구성했습니다.

② 카드와 섹션으로 결과 화면 구성

성격이 다른 정보를 긴 텍스트로 이어 붙이지 않고 카드와 섹션 단위로 나누어 표시했습니다.

점수·감정 지표를 확인하는 영역과 추천 콘텐츠를 탐색하는 영역을 구분해, 분석 결과의 이해와 후속 행동을 연결했습니다.

③ 분석 대기와 결과 표시 단계 구분

분석 요청 중에는 로딩 상태를 제공하고, 분석 완료 후에는 결과 상세 화면으로 연결했습니다.

응답 데이터 표시뿐 아니라 응답을 기다리는 단계까지 화면 흐름에 포함해, 사용자 입력 이후의 대기 과정을 안내했습니다.

④ 신규 결과 확인과 이전 결과 재조회 연결

분석 직후의 결과 상세 화면을 내 분석 결과 목록과 연결했습니다.

최초 결과 확인 이후에도 목록에서 기존 분석 결과를 다시 확인할 수 있는 조회 흐름을 구성했습니다.

[결과]

감정 지표 확인, 총평 읽기, 추천 콘텐츠 탐색으로 이어지는 결과 화면을 구현했습니다. 신규 분석 결과 확인과 기존 분석 결과 재조회 모두를 지원하도록 연결했습니다.

[배운점]

AI 기능을 사용자에게 전달하려면 응답 데이터의 정확성뿐 아니라 정보의 순서와 대기·완료 상태를 함께 설계해야 한다는 점을 배웠습니다. 서버가 반환하는 구조를 사용자가 이해하고 행동할 수 있는 구조로 바꾸는 것이 프론트엔드의 역할이라고 생각합니다.

사용자 매뉴얼까지 연결한 전달 과정

구현과 배포 이후 사용자 매뉴얼을 작성했습니다. 그림과 일기 입력, 분석 결과 확인 등 실제 사용 순서를 문서로 설명하며, 화면의 구성과 서비스의 이용 절차를 함께 전달했습니다. 디자인·구현·배포·안내 문서를 연결해 서비스의 핵심 흐름을 사용할 수 있는 형태로 정리했습니다.

띠링(DDIRING) (2026.02 ~ 2026.06)

어르신과 보호자의 일상 연결을 돕고, 안부 확인 과정을 디지털화한 **AI 기반 일상 돌봄 모바일 앱 서비스**

사용자는 보호자와 어르신 역할로 나뉘어 서비스를 이용하며, 출석 체크, 맞춤 설문, 사진 공유, 알림, 주간 리포트 기능을 통해 비대면 상황에서도 일상 상태를 확인할 수 있습니다.

GitHub | https://github.com/ddiring-team/FRONT

Figma | 띠링 Wireframe

APK | Android 테스트용 APK

Demo | 띠링 시연(홍보) 영상

Team | Plan(1) / Frontend Developer(1) / Backend Developer(1) / AI Engineer(1)

사용기술

- React Native, Expo, TypeScript
- Axios, REST API
- JWT, Secure Store, Kakao Login WebView
- Firebase Cloud Messaging
- S3 Presigned URL
- EAS Build, Android Test Build

핵심 구현 내용

- 역할별 홈·가족방·출석·설문·사진 피드·마이페이지 구현
- 카카오 로그인 WebView, JWT 인증, Secure Store 기반 로그인 상태 관리
- FCM 토큰 발급 및 서버 연동, 이미지 선택 및 S3 직접 업로드
- Android 테스트용 APK와 시연 영상 제작

1. 보호자와 어르신의 목적에 따른 역할별 UX 설계

[문제 상황]

띠링은 하나의 앱을 두 사용자 집단이 이용하지만 필요한 정보와 행동은 달랐습니다. 보호자는 가족방과 활동 현황을 관리하고 결과를 확인해야 했고, 어르신은 출석·설문 응답·사진 공유를 수행해야 했습니다. 같은 기능을 모두 같은 비중으로 배치하면 각 역할의 핵심 행동을 안내하기 어려웠습니다.

[해결 과정]

① 역할별 핵심 행동과 필요한 정보 정리

보호자는 가족방 관리와 활동 상태·설문 결과·사진 피드·리포트 확인이 필요했고, 어르신은 출석 체크·설문 응답·사진 업로드를 수행해야 했습니다.

각 역할의 목적을 기준으로 필요한 화면과 연결 관계를 정리했습니다. 같은 서비스를 이용하더라도 홈 화면에서 먼저 보여 줄 정보와 행동을 구분했습니다.

② Figma에서 역할별 화면 구조 설계

보호자와 어르신의 주요 사용 흐름을 분리해 화면을 설계했습니다.

어르신 화면은 버튼과 안내 문구를 단순하게 구성했습니다. 보호자 화면은 상태 확인과 관리 기능을 빠르게 찾을 수 있도록 정보 구조를 정리했습니다.

③ 화면 설계를 실제 모바일 기능으로 구현

React Native로 역할별 홈 화면과 가족방 초대·참여, 출석 체크, 설문 작성·응답, 사진 피드, 마이페이지를 구현했습니다.

각 화면의 행동을 관련 API와 연결해, 역할에 따른 화면 구성이 실제 기능 수행으로 이어지도록 했습니다.

④ Android 테스트 산출물로 사용 흐름 확인

Android 테스트용 APK와 시연 영상을 제작해, 설계한 화면을 실제 앱에서 확인할 수 있는 산출물로 구성했습니다.

[결과]

같은 서비스 안에서 어르신은 일상 활동을 수행하고 보호자는 관련 상태를 확인·관리할 수 있는 역할별 모바일 흐름을 구현했습니다. 홈 화면과 주요 기능의 연결 구조를 사용자 목적에 맞게 구성했습니다.

[배운점]

역할별 UX는 메뉴를 다르게 보여 주는 수준을 넘어 각 사용자가 무엇을 먼저 해야 하는지 결정하는 설계라는 점을 배웠습니다. 디자인과 프론트엔드를 함께 담당하며 정보 구조를 실제 기능 흐름까지 일관되게 연결했습니다.

2. 권한·디바이스 토큰·서버 연동을 연결한 알림 구현

[문제 상황]

보호자가 어르신의 활동을 놓치지 않도록 모바일 알림을 제공해야 했습니다. 알림은 화면 구현만으로 끝나지 않고, 기기 권한과 디바이스 토큰, 서버 연동이 함께 연결되어야 하는 기능이었습니다.

[해결 과정]

① 기기와 서버 사이의 연동 단계 구분

알림 기능에 필요한 모바일 권한, 디바이스 토큰 발급, 서버 전달을 각각 확인할 대상으로 구분했습니다.

화면 구현만으로 알림 연동을 완료할 수 없으므로, 기기에서 시작한 정보가 서버까지 전달되는 흐름을 구성했습니다.

② FCM 토큰 발급 및 사용자 정보와의 서버 연동

Firebase Cloud Messaging을 활용해 푸시 알림 토큰을 발급받았습니다.

발급한 토큰을 사용자 정보와 함께 서버에 전달하는 API를 연동했습니다. 앱에서 확보한 기기 토큰이 서버의 알림 기능에 사용될 수 있도록 연결했습니다.

③ 실제 Android 기기에서 연동 흐름 확인

실제 기기에서 권한 요청, 토큰 발급, 관련 API 응답 흐름을 반복적으로 확인했습니다.

기기 권한과 외부 알림 서비스가 개입하는 기능이므로, Android 테스트 빌드에서 확인한 흐름을 결과에 포함했습니다.

[결과]

모바일 앱의 토큰 발급과 서버 전달을 연결해 보호자 알림 기능을 구현했습니다. Android 테스트 빌드에서 관련 권한·토큰·API 흐름을 확인했습니다.

[배운점]

외부 서비스 연동에서는 SDK 적용뿐 아니라 기기에서 시작한 정보가 서버 기능에 전달되는 전체 경로를 확인해야 한다는 점을 배웠습니다.

3. 이미지 선택부터 저장까지 연결한 S3 직접 업로드

[문제 상황]

어르신이 사진을 공유하려면 기기에서 이미지를 선택한 뒤 업로드하고, 관련 서버 응답을 처리하는 흐름이 필요했습니다. 화면에서 사진을 선택하는 과정과 실제 저장소로 파일을 전송하는 과정을 함께 연결해야 했습니다.

[해결 과정]

① 기기에서 공유할 이미지 선택

Expo Image Picker를 활용해 사용자가 사진을 선택하는 기능을 구현했습니다.

사진 선택에 필요한 기기 권한과 이미지 선택 흐름을 실제 Android 기기에서 확인했습니다.

② 서버에서 Presigned URL 발급받기

이미지를 전송할 S3 Presigned URL을 서버에서 받는 API를 연동했습니다.

업로드 주소를 받는 서버 요청과 실제 파일을 저장소로 전송하는 요청을 별도 단계로 연결했습니다.

③ 발급된 URL을 이용해 S3에 직접 업로드

서버에서 받은 Presigned URL을 이용해 선택한 이미지를 S3로 직접 업로드했습니다.

사진 선택 → 업로드 URL 발급 → 파일 전송으로 이어지는 흐름을 구현해 모바일 사진 공유 기능에 연결했습니다.

④ 실제 기기에서 파일 전송 흐름 확인

Android 기기에서 권한 요청, 이미지 업로드, API 응답 흐름을 반복적으로 확인했습니다.

테스트용 APK와 데모 영상으로 사진 공유를 포함한 앱의 사용 흐름을 확인할 수 있도록 구성했습니다.

[결과]

사진 선택과 S3 업로드를 연결해 모바일 사진 공유 기능을 구현했습니다. 화면뿐 아니라 권한 요청과 파일 전송을 포함한 흐름을 실제 기기에서 확인했습니다.

[배운점]

미디어 업로드는 화면·기기·서버·저장소가 연결되는 기능이라는 점을 배웠습니다. 단계별로 무엇을 요청하고 어떤 응답을 받아야 하는지 구분하는 것이 기능을 이해하고 검증하는 기준이 되었습니다.